Universidad de los Andes

ISIS-2111 PLE
Project 1
Fecha: August 18, 2026

Objectilang

Throughout the course we are going to implement a programming language covering different characteristics of the topics covered in class. This language is called Objectilang

Objectilang is an object-oriented programming language with generics. Objectilang programs are defined by the objects that compose them, and their interactions. Within objects, it is possible to define values, variables and methods similar to other classic OO languages like Java or C#. In general, Objectilang programs contain the following components:

- Objects
- Variables
- Values
- Methods, and method calls
- Basic types (*e.g.,* Integers, Booleans, Strings)
- Arithmetic, Boolean, and string expressions

Objectilang programs are made up of multiple objects defining their behavior and a main method to execute the program. To enable reuse and incremental software development, Objectilang objects may extend other objects and be defined with generic specifiers, and the relations within them.

Consider the example in Snippet 1 for Objectilang object definitions together with a generic list of dog animals as a valid Objectilang program.

In this simple example the objects `Dog`, `Cat`, and `Animal` objects are defined without any behavior associated to them and are then defined to be the members of different lists.

Behavior is associated with objects by means of a `method` abstraction followed by the full name description of the object. For example, to add the `talk` function to cats, we would define it as in Snippet 2. Note that is also posible to override the behavior of methods already defined, or specified elsewhere in the program. The syntax to do this explicitly uses the `override` construct inside the construction of an object (Line 12 in Snippet 3).

Additionally, objects extend the behavior of other objects by means of the `extends` abstraction. Moreover, to enable reuse, objects may be defined with generic types by adding the `[T]` to the definition of objects (as in Line 5 or Snippet 1). In Objectilang generics may have two modifiers, either `+` or `-`, to represent covariance or contravariance, respectively.

```
object Animal
object Dog extends Animal
object Cat extends Animal

object List[+A](val item: A)

method main {
  val listOfDogs: List[Dog] = new List(new Dog)

  val listOfAnimals: List[Animal] = listOfDogs

  print("Print the line of animal dogs: ${listOfAnimals.item}")
}
```

Snippet 1: Objectilang program of object creation

```
Cat.talk = method() : String {
  print("Cats meows")
}
```

Snippet 2: Objectilang program of object creation

1 Objectilang examples

As example programs of the use of Objectilang we provide the following definitions (in Snippet 3) of the different kinds of abstractions available to the system, and the programs we could write in the language

2 Reserved words

Based on the language examples, the following are reserved words of the language

object	extends	method	val	var
new	override			

In addition to the regular controlflow expressions `if-then-else` and `for`. For loops receive a range of values to iterate over, or a collection of elements.

```
object Animal
object Dog extends Animal
object Puppy extends Dog

object Food
object DogFood extends Food

object Transformer[-In, +Out]
Transform.transform = method(input: In): Out

method main {
  val animalFeeder: Transformer[Animal, DogFood] = new Transformer[
     Animal, DogFood] {
    override transform(animal: Animal): DogFood = {
      println("Preparing specialized food for an animal...")
      new DogFood
    }
  }

  // Contravariance (-In):  Animal is a supertype of Puppy (Accepts
      broader input)
  // Covariance (+Out):     DogFood is a subtype of Food   (Returns
      narrower output)
  val puppyFeeder: Transformer[Puppy, Food] = animalFeeder

  val myPuppy = new Puppy
  val resultantFood: Food = puppyFeeder.transform(myPuppy)

  print("Resulting food type: ${resultantFood.getClass.
     getSimpleName}")
}
```

Snippet 3: Objectilang examples

```
val max = if a > b then a else b
for ((name,rating) <- ratings) print("Movie: $name, Rating: $rating
   ")
```

Snippet 4: Objectilang controlflow examples

3 Identifiers

Identifiers are defined as any sequence of letters, and digits, that begin with a letter, as long as they are not already reserved words.

4 Literals

The following are the symbols used as operators and punctuation symbols valid for Objectilang programs to operate between elements of the appropriate type.

*	/	-	+	**	%
<	>	<=	>=	<>	=
and	or	neg	->	in	=>
.	(	)	[	]	:

Note that this is an initial definition of Objectilang which you will need to grow as more concepts, features, and restrictions are introduced. Therefore, you should take into account the clean design directives for the language.

Task 1. For the first part of the project, your task is to provide the grammar definition of Objectilang . Your definition should clearly state the sets of non-terminals and symbols (the alphabet) as well as the distinguished element of the non-terminals and the production rules.

To define the production rules you should use EBNF notation (where non-terminals are defined as barewords and terminals are defined between quotes ("")).

IMPORTANT: Hand-in a pdf file, with the names of the two students in your group, and the complete definition of your grammar.

Only one person of each group should hand in the project, submissions from multiple sources will be penalized.